Unit-6
RESTful APIs

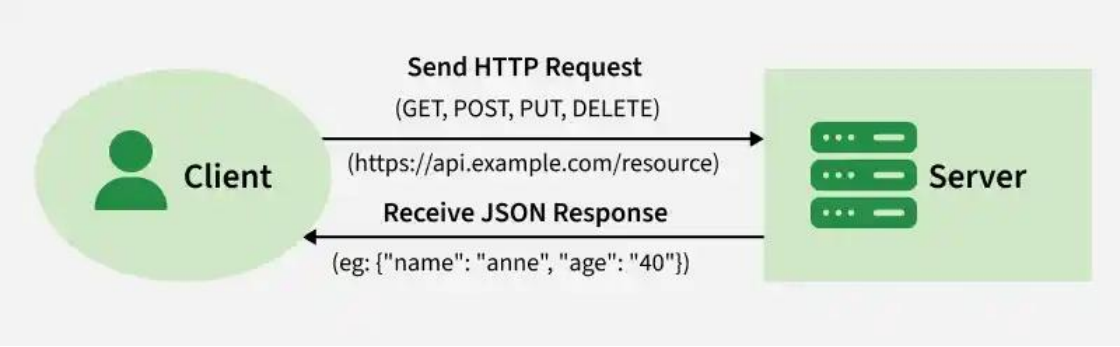
Introduction to API:

- APIs, or Application Programming Interfaces, are the invisible backbone of modern software development.
- They enable applications and systems to communicate and share data efficiently.
- They enable seamless interaction between applications, platforms, and services.
- They power everyday features such as weather updates, payments, and e-commerce checkouts.
- Types of API Architectures
- API architectures define how systems communicate and exchange data, each offering different levels of flexibility, performance, and structure based on application needs.

1. REST (Representational State Transfer):
REST is a simple, flexible API architecture that uses HTTP methods (GET, POST, PUT, DELETE) for communication.
Data Format: JSON, XML.
2. SOAP (Simple Object Access Protocol):
SOAP is a more rigid protocol that requires XML based messaging for communication. Strict and secure protocol using XML for structured messaging.
Data Format: XML

Introduction to RESTful API:

- REST is an acronym for REpresentational State Transfer and an architectural style for distributed hypermedia systems. Roy Fielding first presented it in 2000 in his famous dissertation. Since then, it has become one of the most widely used approaches for building web-based APIs (Application Programming Interfaces).
- REST is not a protocol or a standard, it is an architectural style. During the development phase, API developers can implement REST in a variety of ways.
- Like the other architectural styles, REST also has its guiding principles and constraints. These principles must be satisfied if a service interface is to be referred to as RESTful.



- Uses HTTP methods like GET, POST, PUT, PATCH, and DELETE.
- Client sends requests to server endpoints (URLs).
- Server returns responses such as JSON, XML, HTML, or images.
- Maps HTTP methods to CRUD operations (Create, Read, Update, Delete).

Common HTTP Methods Used in REST API
In REST architecture, the main HTTP methods are GET, POST, PUT and DELETE, which map to CRUD operations. Other less commonly used methods include HEAD and OPTIONS.
1. GET Method
The HTTP GET method retrieves a resource. On success, it returns data (usually JSON or XML) with 200 OK, and on error, it commonly returns 404 Not Found or 400 Bad Request.

```
GET /users/123
```

This request fetches data for the user with ID 123.

2. POST Method
The POST method creates new resources. On success, it returns 201 Created with a Location header pointing to the new resource.

```
POST /users
{
  "name": "Anne",
  "email": "gfg@example.com"
}
```

This request creates a new user with the given data.
Note: POST is neither safe nor idempotent.

3. PUT Method
PUT is used to update or create a resource. It sends the complete resource in the request body and replaces the existing one at the specified URL.

```
PUT /users/123
{
  "name": "Anne",
  "email": "gfg@example.com"
}
```

This request updates the user with ID 123 or creates a new user if one doesn't exist.

4. DELETE Method
It is used to delete a resource identified by a URI. On successful deletion, return HTTP status 200 (OK) along with a response body.

```
DELETE /users/123
```

This request deletes the user with ID 123.

Designing and implementing RESTful APIs using common HTTP methods, such as GET, POST, PUT, and DELETE

- A **REST API** is a web service that allows different applications to communicate over the internet using the **HTTP** protocol. RESTful APIs use standard HTTP methods such as **GET, POST, PUT, and DELETE** to perform operations on resources.
- A **resource** is any type of data that an API manages, such as users, products, or orders.
- Each resource is identified using a **URL (endpoint)**.
  In this example, a simple RESTful API is created using **Flask**. The API manages a list of users and performs operations using HTTP methods.
  Eg.

```
from flask import Flask, request, jsonify

app = Flask(__name__)

data = [
    {"name": "abc", "enrollment_number": 12345, "age": 19},
    {"name": "pqr", "enrollment_number": 67890, "age": 20}
]

# GET
@app.route('/user', methods=['GET'])
def get_user():
    return jsonify(data)

# POST
@app.route('/user', methods=['POST'])
def add_user():
    new_user = request.json
    data.append(new_user)
    return jsonify({"message": "User added", "data": data})

# PUT
@app.route('/user/<int:enrollment_number>', methods=['PUT'])
def update_user(enrollment_number):
    for user in data:
        if user['enrollment_number'] == enrollment_number:
            user['name'] = request.json.get('name', user['name'])
            user['age'] = request.json.get('age', user['age'])
            return jsonify({"message": "User updated", "data": user})

    return jsonify({"message": "User not found"})
```

```
# DELETE
@app.route('/user/<int:enrollment_number>', methods=['DELETE'])
def delete_user(enrollment_number):
    for i in range(len(data)):
        if data[i]["enrollment_number"] == enrollment_number:
            deleted_user = data.pop(i)
            return jsonify({
                "message": "Student deleted successfully",
                "deleted_student": deleted_user
            })

    return jsonify({"message": "Student not found"})

if __name__ == '__main__':
    app.run(debug=True)
```

Tools to Test APIs

- Testing APIs is an important step in development to ensure that the API works correctly and returns the expected results.
- API testing tools allow developers to send HTTP requests such as **GET, POST, PUT, and DELETE** and analyze the responses returned by the server.
- These tools help verify functionality, performance, and reliability of APIs built using standards like **REST** over the **HTTP** protocol.
  Below are some commonly used API testing tools.
  - Postman
  - cURL
  - Insomnia
  - JMeter
  - Swagger UI

Using URLs and resource representations to identify and transfer data in RESTful APIs

A **URL (Uniform Resource Locator)** is used to uniquely identify a resource on the server.
A **resource** is any data object such as a user, product, or order.

Eg.
/users
/users/1
/products
/orders/100

- /users → Represents a collection of users
- /users/1 → Represents a specific user with ID 1

These URLs are also called **API endpoints**.

Structure of a REST API URL

http://example.com/api/users/1
Components:

Component	Meaning
http	Protocol
example.com	Server
/api	API base path
/users	Resource
/1	Specific resource ID

Resource representation means **how the resource data is sent between client and server**.
REST APIs usually use **JSON (JavaScript Object Notation)** format.
Eg:

```
{
"name": "abc",
"enrollment_number": 12345,
"age": 19
}
```

This JSON object represents a **user resource**.
Multiple resources can be represented as a list:
Eg:

```
[
 {"name":"abc","enrollment_number":12345,"age":19},
 {"name":"pqr","enrollment_number":67890,"age":20}
]
```

Working Process

```
Client (Browser/Postman)
        |
        | Request using URL
        ▼
REST API Server
        |
        | Sends JSON representation
        ▼
Client receives data
```

Implementing best practices for designing and implementing RESTful APIs, such as using HTTP status codes, versioning, and error handling
Designing a good RESTful API is not only about making endpoints work—it's also about making them consistent, understandable, and easy for developers to use. Some important best practices include using proper HTTP status codes, API versioning, and structured error handling.
Using Proper HTTP Status Codes
HTTP status codes help the client understand the **result of a request**. Instead of always returning 200 OK, APIs should return the most appropriate status code.

Common Status Codes

Status Code	Meaning
200 OK	Request successful
201 Created	New resource created
400 Bad Request	Invalid request from client
401 Unauthorized	Authentication required
404 Not Found	Resource not found
500 Internal Server Error	Server-side error

API Versioning

API versioning helps developers **update APIs without breaking existing applications**. When changes are made to the API, a new version can be introduced.

Example URL Versioning

/api/v1/users
/api/v2/users

Eg.

```
@app.route('/api/v1/users')
def users_v1():
    return {"version": "v1", "users": ["abc", "pqr"]}

@app.route('/api/v2/users')
def users_v2():
    return {"version": "v2", "users": [{"name":"abc"}, {"name":"pqr"}]}
```

This allows older applications to keep using **v1**, while newer ones can switch to **v2**.

Proper Error Handling

APIs should provide **clear error messages** so developers know what went wrong.

Eg.

```
from flask import request

@app.route('/users', methods=['POST'])
def add_user():
    data = request.json

    if "name" not in data:
        return {"error": "Name is required"}, 400

    return {"message": "User created"}, 201
```

Building scalable and secure RESTful APIs using common frameworks and libraries Flask or FastAPI

RESTful APIs are commonly built using modern frameworks such as **Flask** and **FastAPI**. These frameworks simplify the process of creating web services that follow REST principles and allow developers to build applications that are **scalable, secure, and easy to maintain**.

Using Frameworks for RESTful APIs
Frameworks like Flask and FastAPI provide built-in tools for:

- Handling HTTP requests and responses
- Routing URLs to specific functions
- Returning data in JSON format
- Managing errors and exceptions

Eg.

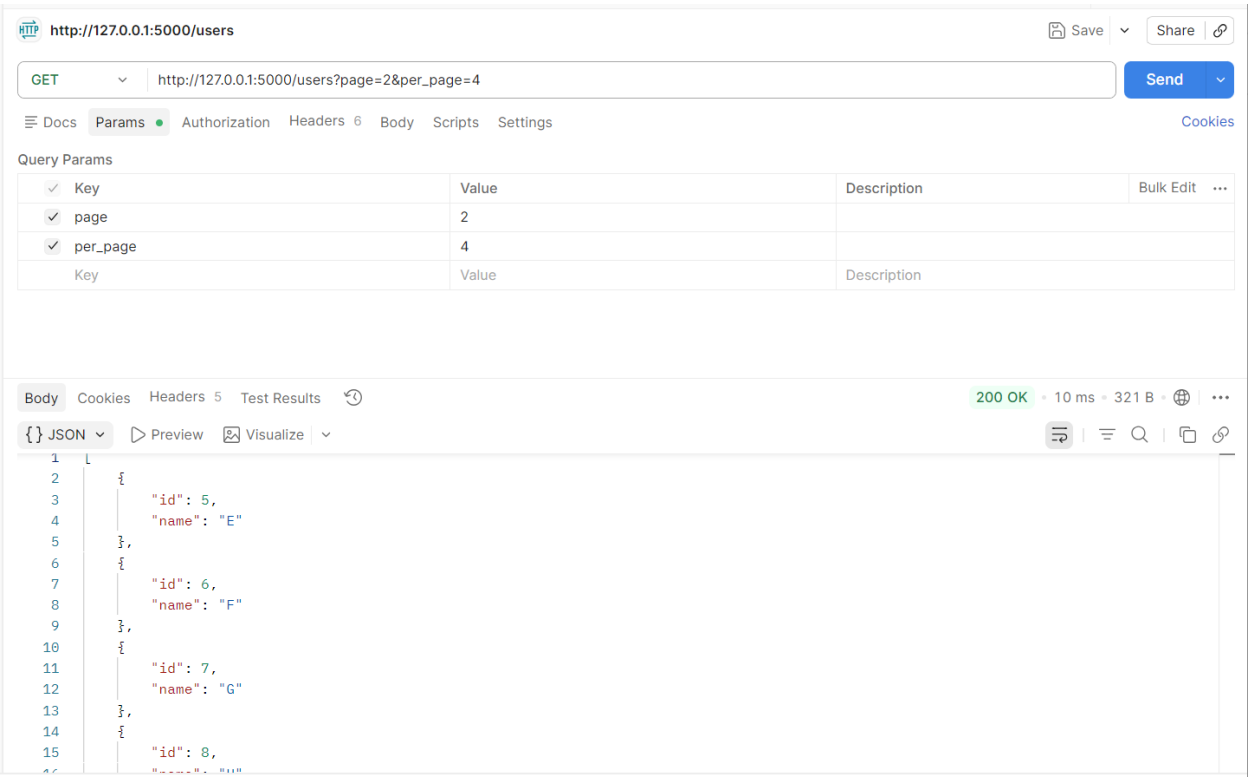
```
from flask import Flask, jsonify

app = Flask(__name__)

@app.route('/users', methods=['GET'])
def get_users():
    users = [
        {"id": 1, "name": "Alice"},
        {"id": 2, "name": "Bob"}
    ]
    return jsonify(users)

if __name__ == "__main__":
    app.run(debug=True)
```

Scalability in RESTful APIs
Scalability refers to the ability of an API to **handle increasing numbers of users and requests efficiently**.
Common techniques include:

Pagination
Instead of returning a large dataset, the API sends data in smaller chunks.
Eg.

```
GET /users?page=1&limit=10
```

- **Efficient Data Handling:** Frameworks allow integration with databases and caching systems to improve performance.
- **Stateless Architecture:** REST APIs are stateless, meaning each request contains all necessary information, which helps servers scale easily.



Security in RESTful APIs

Security is an important aspect when designing APIs. Frameworks provide mechanisms to secure endpoints and protect data.

Authentication: APIs can use tokens such as JSON Web Tokens (JWT) to verify users.

Eg.

Authorization: Bearer <token>

- **Input Validation:** Frameworks help validate incoming data to prevent invalid or malicious inputs.
- **HTTPS Encryption:** Using HTTPS ensures secure communication between client and server.

http://127.0.0.1:5000/profile

Save | Share

GET http://127.0.0.1:5000/profile Send

Docs Params Authorization Headers 9 Body Scripts Settings Cookies

Headers 8 hidden

Key	Value	Description	Bulk Edit Presets
Authorization	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyIjoiY...		
Key	Value	Description	

Body Cookies Headers 5 Test Results 200 OK 9 ms 198 B

{} JSON Preview Visualize


```
{
    "message": "Welcome admin"
}
```